

Universidad Privada Domingo Savio

Tecnología Web II

Entrega 8B

API Routes y UI del Modulo de Horarios

FASE 4: Horarios, Asistencias y Configuración de Evaluación

Objetivos de Aprendizaje

- Crear un esquema de validación Zod para horarios con transformaciones y restricciones personalizadas.
- Implementar API Routes CRUD completas para la gestión de horarios (GET, POST, PUT, DELETE).
- Construir un componente de tabla con Dialog reutilizable para crear y editar horarios (patron de la Entrega 5B).
- Implementar una vista semanal tipo grilla que muestre los horarios de forma visual.
- Aplicar el patron de API Routes diferenciadas por rol (admin, docente, estudiante).

Prerrequisitos

- ✓ Entrega 8A completada: tablas horarios, asistencias y configuracion_evaluacion creadas en Supabase.
- ✓ Políticas RLS de la Entrega 8A aplicadas.
- ✓ Navegación del sidebar actualizada con las entradas de Fase 4.
- ✓ Paginas placeholder creadas para horarios (admin, docente, estudiante).

Aspecto	Admin	Docente	Estudiante
API Route	/api/admin/horarios	/api/docente/horarios	/api/estudiante/horarios

Paso 1: Schema de Validacion Zod

Crea el archivo de validacion para horarios. Este schema se usara tanto en el frontend (Dialog) como en el backend (API Route) para validacion dual.

TypeScript - lib/validations/horario.ts (crear archivo nuevo)

```
import { z } from "zod";

export const horarioSchema = z.object({
  materia_id: z
    .string()
    .uuid("Debe seleccionar una materia valida"),

  dia_semana: z
    .enum(
      ["lunes", "martes", "miercoles", "jueves", "viernes", "sabado"],
      {
        errorMap: () => ({ message: "Seleccione un dia valido" })
      }
    ),

  hora_inicio: z
    .string()
    .regex(
      /^[01]\d|2[0-3]:[0-5]\d$/,
      "Formato de hora invalido (HH:MM)"
    ),

  hora_fin: z
    .string()
    .regex(
      /^[01]\d|2[0-3]:[0-5]\d$/,
      "Formato de hora invalido (HH:MM)"
    ),

  aula: z
    .string()
    .max(50, "Maximo 50 caracteres")
    .transform((v) => v.trim())
    .optional()
    .or(z.literal("")),

  modalidad: z
    .enum(["presencial", "virtual", "hibrida"])
    .default("presencial"),
});

.refine(
  (data) => data.hora_fin > data.hora_inicio,
  {
```

```

    message: "La hora de fin debe ser posterior a la hora de inicio",
    path: ["hora_fin"],
  }
};

export type HorarioFormData = z.infer<typeof horarioSchema>;

// Constantes para los selects del formulario
export const DIAS_SEMANA = [
  { value: "lunes", label: "Lunes" },
  { value: "martes", label: "Martes" },
  { value: "miercoles", label: "Miercoles" },
  { value: "jueves", label: "Jueves" },
  { value: "viernes", label: "Viernes" },
  { value: "sabado", label: "Sabado" },
];

export const MODALIDADES = [
  { value: "presencial", label: "Presencial" },
  { value: "virtual", label: "Virtual" },
  { value: "hibrida", label: "Hibrida" },
];

```

Analisis del Schema

Campo	Validacion Zod	Equivalente en BD
materia_id	z.string().uuid()	UUID NOT NULL REFERENCES materias
dia_semana	z.enum([...6 valores])	CHECK dia_semana IN (...)
hora_inicio / hora_fin	z.string().regex(HH:MM)	TIME NOT NULL
aula	z.string().max(50).optional()	TEXT (nullable)
modalidad	z.enum(...).default('presencial')	DEFAULT 'presencial' CHECK(...)
.refine(hora_fin > hora_inicio)	Validacion cruzada entre campos	CONSTRAINT horario_valido

NOTA: z.refine() para validacion cruzada

A diferencia de los CHECK de campo individual, `.refine()` permite comparar MULTIPLES campos entre si. Es el equivalente en Zod del CONSTRAINT horario_valido de PostgreSQL. Si la validacion falla, el error se asigna al campo especificado en path (hora_fin).

Paso 2: API Route Admin - GET y POST

Crea la API Route principal para el admin. El GET lista todos los horarios con JOIN a materias (para mostrar el nombre), y el POST crea un nuevo horario con validacion Zod.

```
TypeScript - app/api/admin/horarios/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";
import { horarioSchema } from "@lib/validations/horario";

// — GET: Listar todos los horarios —
export async function GET() {
  const supabase = await createClient();

  // Verificar autenticacion
  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json(
      { error: "No autenticado" },
      { status: 401 }
    );
  }

  // Verificar rol admin
  const { data: perfil } = await supabase
    .from("usuarios_perfil")
    .select("rol")
    .eq("id", user.id)
    .single();

  if (perfil?.rol !== "admin") {
    return NextResponse.json(
      { error: "Acceso denegado" },
      { status: 403 }
    );
  }

  // Obtener horarios con JOIN a materias
  const { data: horarios, error } = await supabase
    .from("horarios")
    .select(`
      *,
      materias (id, nombre, codigo)
    `)
    .order("dia_semana")
    .order("hora_inicio");
```

```

    if (error) {
      return NextResponse.json(
        { error: error.message },
        { status: 500 }
      );
    }

    return NextResponse.json(horarios);
  }

  // — POST: Crear un nuevo horario —
  export async function POST(request: Request) {
    const supabase = await createClient();

    // Verificar autenticacion
    const { data: { user }, error: authError } = await supabase.auth.getUser();
    if (authError || !user) {
      return NextResponse.json(
        { error: "No autenticado" },
        { status: 401 }
      );
    }

    // Verificar rol admin
    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json(
        { error: "Acceso denegado" },
        { status: 403 }
      );
    }

    // Parsear y validar body
    const body = await request.json();
    const resultado = horarioSchema.safeParse(body);

    if (!resultado.success) {
      return NextResponse.json(
        { error: resultado.error.errors[0].message },
        { status: 400 }
      );
    }

    // Insertar horario
    const { data: nuevoHorario, error } = await supabase

```

```

    .from("horarios")
    .insert({
      materia_id: resultado.data.materia_id,
      dia_semana: resultado.data.dia_semana,
      hora_inicio: resultado.data.hora_inicio,
      hora_fin: resultado.data.hora_fin,
      aula: resultado.data.aula || null,
      modalidad: resultado.data.modalidad,
    })
    .select()
    .single();

    if (error) {
      // Error 23505: violacion UNIQUE
      if (error.code === "23505") {
        return NextResponse.json(
          { error: "Ya existe un horario para esta materia en ese dia y hora" },
          { status: 409 }
        );
      }
      return NextResponse.json(
        { error: error.message },
        { status: 500 }
      );
    }

    return NextResponse.json(nuevoHorario, { status: 201 });
  }
}

```

ADVERTENCIA: Patron de verificacion de rol

Observa que verificamos el rol del usuario en CADA endpoint, no confiamos solo en el RLS. Esto es defensa en profundidad: RLS protege los datos a nivel de BD, pero la API Route agrega una capa de proteccion a nivel de aplicacion con mensajes de error claros.

Paso 3: API Route Admin - PUT y DELETE

Crea la API Route con parametro dinamico [id] para editar y eliminar horarios individuales. Mismo patron que la Entrega 5B con materias.

```
TypeScript - app/api/admin/horarios/[id]/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";
import { horarioSchema } from "@lib/validations/horario";

// — PUT: Editar un horario —
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const supabase = await createClient();

  // Verificar autenticacion + rol admin
  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  const { data: perfil } = await supabase
    .from("usuarios_perfil")
    .select("rol")
    .eq("id", user.id)
    .single();

  if (perfil?.rol !== "admin") {
    return NextResponse.json({ error: "Acceso denegado" }, { status: 403 });
  }

  // Validar body
  const body = await request.json();
  const resultado = horarioSchema.safeParse(body);

  if (!resultado.success) {
    return NextResponse.json(
      { error: resultado.error.errors[0].message },
      { status: 400 }
    );
  }

  // Actualizar
  const { data: horarioActualizado, error } = await supabase
```

```

    .from("horarios")
    .update({
      materia_id: resultado.data.materia_id,
      dia_semana: resultado.data.dia_semana,
      hora_inicio: resultado.data.hora_inicio,
      hora_fin: resultado.data.hora_fin,
      aula: resultado.data.aula || null,
      modalidad: resultado.data.modalidad,
    })
    .eq("id", id)
    .select()
    .single();

    if (error) {
      if (error.code === "23505") {
        return NextResponse.json(
          { error: "Ya existe un horario para esta materia en ese dia y hora" },
          { status: 409 }
        );
      }
      return NextResponse.json({ error: error.message }, { status: 500 });
    }

    return NextResponse.json(horarioActualizado);
  }

  // — DELETE: Eliminar un horario —
  export async function DELETE(
    _request: Request,
    { params }: { params: Promise<{ id: string }> }
  ) {
    const { id } = await params;
    const supabase = await createClient();

    // Verificar autenticacion + rol admin
    const { data: { user }, error: authError } = await supabase.auth.getUser();
    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "Acceso denegado" }, { status: 403 });
    }
  }

```

```
// Eliminar
const { error } = await supabase
  .from("horarios")
  .delete()
  .eq("id", id);

if (error) {
  return NextResponse.json({ error: error.message }, { status: 500 });
}

return NextResponse.json({ mensaje: "Horario eliminado" });
}
```

NOTA: params async en Next.js 16

Recuerda que en Next.js 16, params es una Promise y debe ser await-eada: `const { id } = await params`. Este cambio se introdujo en Next.js 15 y es obligatorio en la version 16. Si olvidas el await, obtendras un error de tipo.

Paso 4: API Route Docente - GET

El docente solo ve los horarios de las materias que tiene asignadas. La consulta filtra por docente_id usando el JOIN implicito de Supabase.

```
TypeScript - app/api/docente/horarios/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  // Obtener IDs de materias del docente
  const { data: misMaterias } = await supabase
    .from("materias")
    .select("id")
    .eq("docente_id", user.id);

  if (!misMaterias || misMaterias.length === 0) {
    return NextResponse.json([]);
  }

  const materiaIds = misMaterias.map((m) => m.id);

  // Obtener horarios de esas materias
  const { data: horarios, error } = await supabase
    .from("horarios")
    .select(`
      *,
      materias (id, nombre, codigo)
    `)
    .in("materia_id", materiaIds)
    .order("dia_semana")
    .order("hora_inicio");

  if (error) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }

  return NextResponse.json(horarios);
}
```

TIP: Patron de consulta en 2 pasos

Primero obtenemos los IDs de materias del docente, luego filtramos horarios con `.in()`. Este patron evita subqueries complejas y es mas legible. Es el mismo que usamos en la Entrega 7A para obtener inscripciones del docente.

Paso 5: API Route Estudiante - GET

El estudiante ve los horarios de las materias en las que esta inscrito (estado = 'inscrito'). Similar al docente, usamos una consulta en 2 pasos.

```
TypeScript - app/api/estudiante/horarios/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  // Obtener materias donde esta inscrito (estado activo)
  const { data: inscripciones } = await supabase
    .from("inscripciones")
    .select("materia_id")
    .eq("estudiante_id", user.id)
    .eq("estado", "inscrito");

  if (!inscripciones || inscripciones.length === 0) {
    return NextResponse.json([]);
  }

  const materiaIds = inscripciones.map((i) => i.materia_id);

  // Obtener horarios de esas materias
  const { data: horarios, error } = await supabase
    .from("horarios")
    .select(`
      *,
      materias (id, nombre, codigo)
    `)
    .in("materia_id", materiaIds)
    .order("dia_semana")
    .order("hora_inicio");

  if (error) {
    return NextResponse.json({ error: error.message }, { status: 500 });
  }

  return NextResponse.json(horarios);
}
```

ADVERTENCIA: Solo materias con estado inscrito

Filtramos por `.eq('estado', 'inscrito')` para que el estudiante no vea horarios de materias retiradas. Si un estudiante se retira de una materia, su horario desaparece automaticamente de la vista.

Paso 6: Componente Admin - HorariosTable

Este es el componente mas extenso de la entrega. Combina una tabla de datos con un Dialog reutilizable para crear y editar, siguiendo el patron de materias-table.tsx de la Entrega 5B.

Crea el archivo components/dashboard/admin/horarios-table.tsx con el siguiente contenido:

TypeScript - components/dashboard/admin/horarios-table.tsx (Parte 1: Imports y Tipos)

```
"use client";

import { useEffect, useState } from "react";
import { createClient } from "@lib/supabase/client";
import { toast } from "sonner";
import { Plus, Pencil, Trash2, Clock, Filter } from "lucide-react";

// UI Components
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import {
  Dialog, DialogContent, DialogDescription,
  DialogHeader, DialogTitle, DialogTrigger,
} from "@components/ui/dialog";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
  AlertDialogTrigger,
} from "@components/ui/alert-dialog";
import { Badge } from "@components/ui/badge";

import {
  horarioSchema,
  DIAS_SEMANA,
  MODALIDADES,
} from "@lib/validations/horario";

// Tipos
interface Materia {
```

```

    id: string;
    nombre: string;
    codigo: string;
  }

interface Horario {
  id: string;
  materia_id: string;
  dia_semana: string;
  hora_inicio: string;
  hora_fin: string;
  aula: string | null;
  modalidad: string;
  materias: Materia;
}

```

TypeScript - horarios-table.tsx (Parte 2: Estado y Carga)

```

export default function HorariosTable() {
  // Estado principal
  const [horarios, setHorarios] = useState<Horario[]>([]);
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [cargando, setCargando] = useState(true);
  const [dialogAbierto, setDialogAbierto] = useState(false);
  const [guardando, setGuardando] = useState(false);

  // Estado del formulario
  const [horarioEditando, setHorarioEditando] = useState<Horario | null>(null);
  const [formData, setFormData] = useState({
    materia_id: "",
    dia_semana: "",
    hora_inicio: "",
    hora_fin: "",
    aula: "",
    modalidad: "presencial",
  });

  // Filtro
  const [filtroDia, setFiltroDia] = useState<string>("todos");

  // Cargar datos al montar
  useEffect(() => {
    cargarDatos();
  }, []);

  async function cargarDatos() {
    setCargando(true);
    try {
      const [resHorarios, resMaterias] = await Promise.all([
        fetch("/api/admin/horarios"),

```

```

    fetch("/api/admin/materias"),
  ]);

  if (resHorarios.ok) {
    const data = await resHorarios.json();
    setHorarios(data);
  }
  if (resMaterias.ok) {
    const data = await resMaterias.json();
    setMaterias(data);
  }
} catch (error) {
  toast.error("Error al cargar datos");
} finally {
  setCargando(false);
}
}

```

TypeScript - horarios-table.tsx (Parte 3: Funciones CRUD)

```

// — Abrir Dialog para CREAR —
function abrirCrear() {
  setHorarioEditando(null);
  setFormData({
    materia_id: "",
    dia_semana: "",
    hora_inicio: "",
    hora_fin: "",
    aula: "",
    modalidad: "presencial",
  });
  setDialogAbierto(true);
}

// — Abrir Dialog para EDITAR —
function abrirEditar(horario: Horario) {
  setHorarioEditando(horario);
  setFormData({
    materia_id: horario.materia_id,
    dia_semana: horario.dia_semana,
    hora_inicio: horario.hora_inicio.slice(0, 5),
    hora_fin: horario.hora_fin.slice(0, 5),
    aula: horario.aula || "",
    modalidad: horario.modalidad,
  });
  setDialogAbierto(true);
}

// — Guardar (crear o editar) —
async function guardarHorario() {
  // Validar con Zod

```

```

const resultado = horarioSchema.safeParse(formData);
if (!resultado.success) {
  toast.error(resultado.error.errors[0].message);
  return;
}

setGuardando(true);
try {
  const esEdicion = horarioEditando !== null;
  const url = esEdicion
    ? `/api/admin/horarios/${horarioEditando.id}`
    : "/api/admin/horarios";

  const res = await fetch(url, {
    method: esEdicion ? "PUT" : "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(resultado.data),
  });

  if (!res.ok) {
    const error = await res.json();
    toast.error(error.error);
    return;
  }

  toast.success(
    esEdicion ? "Horario actualizado" : "Horario creado"
  );
  setDialogAbierto(false);
  cargarDatos();
} catch (error) {
  toast.error("Error de conexion");
} finally {
  setGuardando(false);
}
}

// — Eliminar —
async function eliminarHorario(id: string) {
  try {
    const res = await fetch(`/api/admin/horarios/${id}`, {
      method: "DELETE",
    });

    if (!res.ok) {
      const error = await res.json();
      toast.error(error.error);
      return;
    }
  }
}

```

```

    toast.success("Horario eliminado");
    cargarDatos();
  } catch (error) {
    toast.error("Error al eliminar");
  }
}

```

TypeScript - horarios-table.tsx (Parte 4: Render - Filtro y Tabla)

```

// — Filtrar horarios —
const horariosFiltrados = filtroDia === "todos"
  ? horarios
  : horarios.filter((h) => h.dia_semana === filtroDia);

// — Helper: capitalizar dia —
function capitalizarDia(dia: string) {
  return dia.charAt(0).toUpperCase() + dia.slice(1);
}

// — Badge de modalidad —
function badgeModalidad(modalidad: string) {
  const variantes: Record<string, "default" | "secondary" | "outline"> = {
    presencial: "default",
    virtual: "secondary",
    hibrida: "outline",
  };
  return (
    <Badge variant={variantes[modalidad] || "default"}>
      {capitalizarDia(modalidad)}
    </Badge>
  );
}

if (cargando) {
  return <p className="text-muted-foreground p-6">Cargando horarios...</p>;
}

return (
  <div className="space-y-4">
    {/* Barra de acciones */}
    <div className="flex items-center justify-between">
      <div className="flex items-center gap-2">
        <Filter className="h-4 w-4 text-muted-foreground" />
        <Select value={filtroDia} onChange={setFiltroDia}>
          <SelectTrigger className="w-[180px]">
            <SelectValue placeholder="Filtrar por dia" />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="todos">Todos los dias</SelectItem>
            {DIAS_SEMANA.map((d) => (
              <SelectItem key={d.value} value={d.value}>

```

```

        {d.label}
      </SelectedItem>
    )}}
  </SelectContent>
</Select>
</div>

<Dialog open={dialogAbierto} onOpenChange={setDialogAbierto}>
  <DialogTrigger asChild>
    <Button onClick={abrirCrear}>
      <Plus className="h-4 w-4 mr-2" />
      Nuevo Horario
    </Button>
  </DialogTrigger>

  {/* ... DialogContent en Parte 5 ... */}
</Dialog>
</div>

{/* Tabla */}
<Table>
  <TableHeader>
    <TableRow>
      <TableHead>Materia</TableHead>
      <TableHead>Dia</TableHead>
      <TableHead>Hora Inicio</TableHead>
      <TableHead>Hora Fin</TableHead>
      <TableHead>Aula</TableHead>
      <TableHead>Modalidad</TableHead>
      <TableHead className="text-right">Acciones</TableHead>
    </TableRow>
  </TableHeader>
  <TableBody>
    {horariosFiltrados.map((h) => (
      <TableRow key={h.id}>
        <TableCell className="font-medium">
          {h.materias.codigo} - {h.materias.nombre}
        </TableCell>
        <TableCell>{capitalizarDia(h.dia_semana)}</TableCell>
        <TableCell>{h.hora_inicio.slice(0, 5)}</TableCell>
        <TableCell>{h.hora_fin.slice(0, 5)}</TableCell>
        <TableCell>{h.aula || "-"}</TableCell>
        <TableCell>{badgeModalidad(h.modalidad)}</TableCell>
        <TableCell className="text-right space-x-1">
          <Button
            variant="ghost"
            size="icon"
            onClick={() => abrirEditar(h)}
          >
            <Pencil className="h-4 w-4" />
          </Button>
        </TableCell>
      </TableRow>
    ))}
  </TableBody>
</Table>

```

```

        <AlertDialog>
          <AlertDialogTrigger asChild>
            <Button variant="ghost" size="icon">
              <Trash2 className="h-4 w-4 text-destructive" />
            </Button>
          </AlertDialogTrigger>
          <AlertDialogContent>
            <AlertDialogHeader>
              <AlertDialogTitle>
                Eliminar horario
              </AlertDialogTitle>
              <AlertDialogDescription>
                Se eliminara el bloque de
                {" "}{capitalizarDia(h.dia_semana)}
                {" "}{h.hora_inicio.slice(0, 5)} -
                {" "}{h.hora_fin.slice(0, 5)} de
                {" "}{h.materias.nombre}.
                Esta accion no se puede deshacer.
              </AlertDialogDescription>
            </AlertDialogHeader>
            <AlertDialogFooter>
              <AlertDialogCancel>Cancelar</AlertDialogCancel>
              <AlertDialogAction
                className="bg-destructive text-destructive-foreground"
                onClick={() => eliminarHorario(h.id)}
              >
                Eliminar
              </AlertDialogAction>
            </AlertDialogFooter>
          </AlertDialogContent>
        </AlertDialog>
      </TableCell>
    </TableRow>
  )})
  {horariosFiltrados.length === 0 && (
    <TableRow>
      <TableCell
        colSpan={7}
        className="text-center text-muted-foreground py-8"
      >
        No hay horarios registrados
      </TableCell>
    </TableRow>
  )}
</TableBody>
</Table>
</div>
);
}

```

```

{/* Este bloque va DENTRO del <Dialog> de la Parte 4 */}
<DialogContent className="sm:max-w-[500px]">
  <DialogHeader>
    <DialogTitle>
      {horarioEditando ? "Editar Horario" : "Nuevo Horario"}
    </DialogTitle>
    <DialogDescription>
      {horarioEditando
        ? "Modifica los datos del bloque horario"
        : "Agrega un nuevo bloque horario a una materia"
      }
    </DialogDescription>
  </DialogHeader>

  <div className="grid gap-4 py-4">
    {/* Materia */}
    <div className="grid gap-2">
      <Label htmlFor="materia_id">Materia</Label>
      <Select
        value={formData.materia_id}
        onChange={v =>
          setFormData({ ...formData, materia_id: v })
        }
      >
        <SelectTrigger>
          <SelectValue placeholder="Seleccionar materia" />
        </SelectTrigger>
        <SelectContent>
          {materias.map(m => (
            <SelectItem key={m.id} value={m.id}>
              {m.codigo} - {m.nombre}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>

    {/* Dia de la semana */}
    <div className="grid gap-2">
      <Label htmlFor="dia_semana">Dia de la semana</Label>
      <Select
        value={formData.dia_semana}
        onChange={v =>
          setFormData({ ...formData, dia_semana: v })
        }
      >
        <SelectTrigger>
          <SelectValue placeholder="Seleccionar dia" />
        </SelectTrigger>
        <SelectContent>
          {DIAS_SEMANA.map(d => (
            <SelectItem key={d.value} value={d.value}>

```

```

        {d.label}
      </SelectedItem>
    )))
  </SelectContent>
</Select>
</div>

{/* Horas */}
<div className="grid grid-cols-2 gap-4">
  <div className="grid gap-2">
    <Label htmlFor="hora_inicio">Hora Inicio</Label>
    <Input
      id="hora_inicio"
      type="time"
      value={formData.hora_inicio}
      onChange={(e) =>
        setFormData({
          ...formData,
          hora_inicio: e.target.value,
        })
      }
    />
  </div>
  <div className="grid gap-2">
    <Label htmlFor="hora_fin">Hora Fin</Label>
    <Input
      id="hora_fin"
      type="time"
      value={formData.hora_fin}
      onChange={(e) =>
        setFormData({
          ...formData,
          hora_fin: e.target.value,
        })
      }
    />
  </div>
</div>

{/* Aula y Modalidad */}
<div className="grid grid-cols-2 gap-4">
  <div className="grid gap-2">
    <Label htmlFor="aula">Aula (opcional)</Label>
    <Input
      id="aula"
      placeholder="Ej: Aula 201"
      value={formData.aula}
      onChange={(e) =>
        setFormData({
          ...formData,
          aula: e.target.value,
        })
      }
    />
  </div>
  <div className="grid gap-2">
    <Label htmlFor="modalidad">Modalidad</Label>
    <Input
      id="modalidad"
      type="text"
      value={formData.modalidad}
      onChange={(e) =>
        setFormData({
          ...formData,
          modalidad: e.target.value,
        })
      }
    />
  </div>
</div>

```

```

    }
  />
</div>
<div className="grid gap-2">
  <Label htmlFor="modalidad">Modalidad</Label>
  <Select
    value={formData.modalidad}
    onChange={(v) =>
      setFormData({ ...formData, modalidad: v })
    }
  >
    <SelectTrigger>
      <SelectValue />
    </SelectTrigger>
    <SelectContent>
      {MODALIDADES.map((m) => (
        <SelectItem key={m.value} value={m.value}>
          {m.label}
        </SelectItem>
      ))}
    </SelectContent>
  </Select>
</div>
</div>
</div>

<div className="flex justify-end">
  <Button
    onClick={guardarHorario}
    disabled={guardando}
  >
    {guardando ? "Guardando..." : "Guardar"}
  </Button>
</div>
</DialogContent>

```

Paso 7: Componente Reutilizable - HorarioSemanal

Este componente muestra los horarios en formato de grilla semanal, similar a un calendario de clases. Lo usaran tanto docentes como estudiantes. Es un componente de solo lectura (no edita datos).

TypeScript - components/dashboard/horario-semanal.tsx (crear archivo nuevo)

```
"use client";

import { Badge } from "@components/ui/badge";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Clock } from "lucide-react";

interface Materia {
  id: string;
  nombre: string;
  codigo: string;
}

interface Horario {
  id: string;
  dia_semana: string;
  hora_inicio: string;
  hora_fin: string;
  aula: string | null;
  modalidad: string;
  materias: Materia;
}

interface HorarioSemanalProps {
  horarios: Horario[];
  titulo?: string;
}

const DIAS_ORDEN = [
  "lunes", "martes", "miercoles",
  "jueves", "viernes", "sabado",
];

const DIAS_LABELS: Record<string, string> = {
  lunes: "Lun", martes: "Mar", miercoles: "Mie",
  jueves: "Jue", viernes: "Vie", sabado: "Sab",
};

const MODALIDAD_COLORS: Record<string, string> = {
  presencial: "bg-blue-100 border-blue-300 text-blue-800",
  virtual: "bg-green-100 border-green-300 text-green-800",
}
```

```

    hibrida: "bg-purple-100 border-purple-300 text-purple-800",
  };

export default function HorarioSemanal({
  horarios,
  titulo = "Mi Horario Semanal",
}: HorarioSemanalProps) {
  // Agrupar horarios por dia
  const horariosPorDia = DIAS_ORDEN.map((dia) => ({
    dia,
    label: DIAS_LABELS[dia],
    bloques: horarios
      .filter((h) => h.dia_semana === dia)
      .sort((a, b) => a.hora_inicio.localeCompare(b.hora_inicio)),
  }));

  if (horarios.length === 0) {
    return (
      <Card>
        <CardContent className="py-12 text-center">
          <Clock className="h-12 w-12 mx-auto text-muted-foreground mb-4" />
          <p className="text-muted-foreground">
            No hay horarios registrados
          </p>
        </CardContent>
      </Card>
    );
  }

  return (
    <Card>
      <CardHeader>
        <CardTitle className="flex items-center gap-2">
          <Clock className="h-5 w-5" />
          {titulo}
        </CardTitle>
      </CardHeader>
      <CardContent>
        <div className="grid grid-cols-6 gap-2">
          {/* Header de dias */}
          {horariosPorDia.map(({ dia, label }) => (
            <div
              key={dia}
              className="text-center font-semibold text-sm py-2
                bg-muted rounded-t-md"
            >
              {label}
            </div>
          ))}

          {/* Bloques por dia */}
          {horariosPorDia.map(({ dia, bloques }) => (

```

```

<div key={dia} className="space-y-1 min-h-[100px]">
  {bloques.map((bloque) => (
    <div
      key={bloque.id}
      className={`
        p-2 rounded-md border text-xs
        ${MODALIDAD_COLORS[bloque.modalidad]}
      `}
    >
      <p className="font-semibold truncate">
        {bloque.materias.codigo}
      </p>
      <p className="truncate">
        {bloque.materias.nombre}
      </p>
      <p className="mt-1">
        {bloque.hora_inicio.slice(0, 5)} -
        {" "}{bloque.hora_fin.slice(0, 5)}
      </p>
      {bloque.aula && (
        <p className="text-xs opacity-75">
          {bloque.aula}
        </p>
      )}
    </div>
  ))}
  {bloques.length === 0 && (
    <div className="text-center text-muted-foreground
      text-xs py-4">
      --
    </div>
  )}
</div>
  ))}
</div>

{/* Leyenda de modalidad */}
<div className="flex gap-4 mt-4 text-xs text-muted-foreground">
  <div className="flex items-center gap-1">
    <div className="w-3 h-3 rounded bg-blue-200 border border-blue-300" />
    Presencial
  </div>
  <div className="flex items-center gap-1">
    <div className="w-3 h-3 rounded bg-green-200 border border-green-300" />
    Virtual
  </div>
  <div className="flex items-center gap-1">
    <div className="w-3 h-3 rounded bg-purple-200 border border-purple-300" />
    Hibrida
  </div>
</div>
</CardContent>
</Card>

```

```
);  
}
```

TIP: Componente reutilizable con props

HorarioSemanal recibe horarios[] como prop, no los busca internamente. Esto lo hace reutilizable: el docente le pasa SUS horarios, el estudiante le pasa los SUYOS, ambos usan el mismo componente visual. El titulo tambien es personalizable.

Paso 8: Actualizar Paginas

Reemplaza las paginas placeholder creadas en la Entrega 8A con las implementaciones reales:

TypeScript - app/(dashboard)/admin/horarios/page.tsx (reemplazar)

```
import HorariosTable from "@components/dashboard/admin/horarios-table";

export default function HorariosAdminPage() {
  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">
        Gestion de Horarios
      </h1>
      <HorariosTable />
    </div>
  );
}
```

TypeScript - app/(dashboard)/docente/horarios/page.tsx (reemplazar)

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import HorarioSemanal from "@components/dashboard/horario-semanal";

export default function HorariosDocentePage() {
  const [horarios, setHorarios] = useState([]);
  const [cargando, setCargando] = useState(true);

  useEffect(() => {
    async function cargar() {
      try {
        const res = await fetch("/api/docente/horarios");
        if (res.ok) {
          setHorarios(await res.json());
        }
      } catch {
        toast.error("Error al cargar horarios");
      } finally {
        setCargando(false);
      }
    }
    cargar();
  }, []);

  if (cargando) {
```

```

    return <p className="p-6 text-muted-foreground">Cargando...</p>;
  }

  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">Mis Horarios</h1>
      <HorarioSemanal
        horarios={horarios}
        titulo="Horario de Mis Materias"
      />
    </div>
  );
}

```

TypeScript - app/(dashboard)/estudiante/horarios/page.tsx (reemplazar)

```

"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import HorarioSemanal from "@components/dashboard/horario-semanal";

export default function HorariosEstudiantePage() {
  const [horarios, setHorarios] = useState([]);
  const [cargando, setCargando] = useState(true);

  useEffect(() => {
    async function cargar() {
      try {
        const res = await fetch("/api/estudiante/horarios");
        if (res.ok) {
          setHorarios(await res.json());
        }
      } catch {
        toast.error("Error al cargar horarios");
      } finally {
        setCargando(false);
      }
    }
    cargar();
  }, []);

  if (cargando) {
    return <p className="p-6 text-muted-foreground">Cargando...</p>;
  }

  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">Mi Horario</h1>
      <HorarioSemanal

```

```
        horarios={horarios}
        titulo="Horario de Materias Inscritas"
      />
    </div>
  );
}
```

Verificación Final

Verifica cada punto antes de continuar con la Entrega 9A:

- ✓ lib/validations/horario.ts existe y exporta horarioSchema, HorarioFormData, DIAS_SEMANA y MODALIDADES.
- ✓ API Route /api/admin/horarios responde GET (lista) y POST (crear) correctamente.
- ✓ API Route /api/admin/horarios/[id] responde PUT (editar) y DELETE (eliminar) correctamente.
- ✓ API Route /api/docente/horarios responde GET con solo las materias del docente.
- ✓ API Route /api/estudiante/horarios responde GET con solo materias inscritas.
- ✓ La tabla admin muestra horarios con filtro por dia, Dialog para crear/editar, y AlertDialog para eliminar.
- ✓ La vista semanal se renderiza correctamente para docente y estudiante con colores por modalidad.
- ✓ El toast de Sonner muestra mensajes de exito y error en cada operacion.
- ✓ El error 409 (duplicado) se maneja correctamente al intentar crear horarios identicos.
- ✓ npm run dev funciona sin errores en la consola del navegador.

Ejercicio Propuesto

Completa estas 4 tareas para reforzar lo aprendido:

1. Tarea 1: Agrega un filtro por materia al componente HorariosTable del admin (Select adicional junto al filtro de dia). Pista: usa el mismo array materias que ya cargas con Promise.all.
2. Tarea 2: Modifica el componente HorarioSemanal para que al hacer click en un bloque se muestre un tooltip o popover con informacion detallada (nombre completo, docente, creditos).
3. Tarea 3: Agrega una columna 'Cantidad de Bloques' a la tabla de materias del admin (Entrega 5A) que muestre cuantos bloques horarios tiene cada materia. Pista: usa un JOIN count.
4. Tarea 4: Crea un endpoint GET /api/admin/horarios/disponibilidad que reciba un dia_semana y hora_inicio como query params y devuelva las aulas que estan ocupadas en ese bloque. Esto es util para evitar conflictos de aulas.

Proximo: Entrega 9A

En la Entrega 9A implementaremos el modulo de Asistencias (lado docente): API Routes para registrar asistencia masiva por materia y fecha, componente de 'pasar lista' con tabla de estudiantes inscritos y selector de estado, e integracion con la tabla asistencias creada en la Entrega 8A.

Que vamos a construir	Archivos nuevos
API Route: GET + POST asistencias (docente)	app/api/docente/asistencias/route.ts
API Route: PUT asistencia individual	app/api/docente/asistencias/[id]/route.ts
Componente: pasar lista por materia	components/dashboard/docente/pasar-lista.tsx
Componente: historial de asistencias	components/dashboard/docente/historial-asistencias.tsx